

**SHELL / BASH**

**Files:** `ls -ladc ..pwdmkdir -pcp -rmvrm -rftouchrmdir` (empty only).

**Text:** `catgrep PAT fsed 's/A/B/g' tr a-z A-Z (stdin) sort -nruuniq -c (adja-cent!)wc -lwcucut -d: -f1awk '{print $2}'head/tail -ntail -fcomm(sorted).`

**Perms:** `chmod +x; 755 exec, 644 file, 600 secret. which, type -a, export VAR=v, source f=. f.`

**I/O:** `>` overwrite, `>>` append, `2>` err, `2>&1` err→out, `<` stdin, `|` pipe, `<EOF` heredoc, `<<` herestring, `<(cmd)` proc-sub.

**Exit:** 0=ok, 1=err, 2=misuse, 126=not exec, 127=not found, 130=Ctrl-C. `$?`=last.

**Vars:** no space around =. Always `"$VAR". ${1:-def}, ${1:=def}, ${p###/} basename, ${f%.*} noext, ${s/A/B}, ${s//A/B}` all.

**Test [ ]:** `-f` file, `-d` dir, `-z` empty, `-eq` `-ne` `-lt` `-gt` int, `=` != str, `!` not. `[[ ]]` = bash: no word-split, `&& |` =~ regex inside.

**Control:** `for i in 1 2 3; do .. done; for ((i=0;i<n;i++)); while [ ]; do .. done; case x in a) ;; *) ;; esac; break; continue.`

**Func:** `f(){ local x=$1; echo $x; }; return = exit 0-255.`

**Special:** `$0` name, `$1..$9` args, `##` count, `"@"` all-quoted, `$$` pid, `!` bg pid, `$(cmd)` sub, `$( (1+2) )` arith, `cmd & bg, jobs/fg/bg/wait`.

**Shebang:** `#!/bin/bash Debug: bash -n syntax, -x trace, -v raw; set -e (suppressed in && | ||), set -u, set -o pipefail`.

**Brace:** `{a,b,c}, {1..5}` (textual, before all expansions).

**Gotchas:** `sort f > f` clobbers; useless `cat`; single-quoted `'$VAR'` no expand; `glob * ≠` regex `*`.

**REGULAR EXPRESSIONS**

**Classes:** `[abc]`, `[a-z]`, `[^0-9]` neg, `.` any (not `\n`); `\d \D \w \W \s \S`.

**Quant (greedy):** `*0+; +1+; ?0-1; {n}; {n,m}`. Add `?` for lazy: `*? +? {n,m}?`.

**Anchor:** `^` start, `$` end, `\b` word-bnd, `\B` not. Zero-width.

**Groups:** `(x)` capture, `(?:x)` non-cap, `(?<n>x)` named, `(a|b)` alt, `\1` backref.

**Lookaround (0-width):** `(?=x)`, `(?!x)`, `(?<=x)`, `(?<!x)`.

**Examples:** `^[A-Za-z0-9]{8,}$` alnum pwd `≥ 8; <. *?>` lazy tag.

**Glob ≠ Regex:** `glob *=` any-str, `?=1` ch, `[ab]`; `regex *=` "0+ of prev".

**Bash:** `[[ $s =~ ^[0-9]+$ ]]`. Python: always raw `r"\d+"`.

**Pitfall:** `.` skips `\n` unless DOTALL/s flag; greedy by default; backref needs capturing group.

**PYTHON**

**Typing:** dynamic + strong (no "5"+3). Indent=scope (4sp). LEGB resolution; `nonlocal`/`global`.

**Ops:** `/` float, `//` floor, `**` pow, `^` XOR (!).

**Str:** f-str `f"{x:.2f}"`; `.strip` `.split` `.replace` `.startswith`; `"".join(lst)`.

**Slice:** `s[a:b:c]`, neg-idx, `s[::-1]` reverse. Shallow copy.

**Coll:** list `[]`, tuple `()`, set `{x}`, dict `{k:v}`; `d.get(k,def)`, `d.items()`. Comp `[x*x for x in r if x>0]`; generator `(...)` lazy.

**Iter:** `for x in xs, enumerate, zip, range(a,b,c)`.

**Func:** `def f(a,b=1,*args,**kw)->int:`. Doc `"""..."""`. **Trap:** mutable default `def f(x=[]):` use `None`.

**OOP:** `class C: def __init__(self):, __str__, __priv` convention. Duck typing.

**Exc (EAFF):** `try: ... except KeyError: ...` Never bare `except:`. Common: `Value/Type/Index/Key/FileNotFound/ZeroDiv`.

**I/O:** with `open(f)` as `fh:` for line in `fh: ...` (auto-close).

**CLI:** `sys.argv`; `print(...,file=sys.stderr)`; `sys.exit(1); argparse`.

**Regex:** `re.search/findall/sub;m.group(n|name)`.

**raw r""...**

**Truthy:** if not items: empty. Use `is None`, never `==None`. `None/0/""/[ ]/{}` falsy.

**Style:** `snake_case`, `PascalCase`, `UPPER_CONST` (PEP 8).

**JAVASCRIPT / NODE.JS**

**Decl:** `let` (block, reassign), `const` (block, no reassign), **never var**.

**Eq:** `===` strict no coerce; `==` coerces (`0==false!`). Use `==!/!=`.

**Nothing:** `undefined` = uninit, `null` = intentional. Optional `?`, nullish `??`.

**Event loop:** 1-thread + non-blocking I/O. Call stack → task queue → event loop. `setTimeout(f,0)` = after stack empty. **Block main = freeze app**.

**Async:** callback → Promise → `async/await`. `await` only inside `async`; wrap `try/catch`. **Trap:** `arr.forEach(async f)` ignores promises; use `for...of (seq)` or `Promise.all(arr.map(...))` (par).

**Arrow:** `(a,b)=>a+b`; lexical `this`.

**Destr:** `const {a,b}=0, const [x,y]=arr`, default `{a=1}, rest ...rest, spread {...o,b:2}`.

**Arrays:** `.map` `.filter` `.reduce` `.find` `.some` `.every` `.includes`.

**Tmpl:** `'hi ${name}'`.

**Node:** `require('m')` / ESM `import`; `fs.readFile(p,cb)` `async` vs `readFileSync` blocks.

**Express:** `app.get('/u/:id', (req,res)=>res.json(req.params/query/body)`.

**NPM:** `npm init`, install pkg, `install -D`, run script.

**Hoist:** `var`+funcs hoisted; `let/const` in TDZ until decl.

**REACT**

**Paradigm:** declarative; `UI = f(state)`; re-render on state change.

**Components:** capitalized func returning JSX; props read-only.

**JSX:** `className` (not class), `htmlFor`, `onClick` camel; self-close `<br/>`; inline `style={{color:'red'}}`; fragment `<>...</>`; `expr {x}`.

**useState:** `const [v,setV]=useState(0)`. Functional `setV(p=>p+1)` avoids stale closures. Batched in React 18+.

**useEffect:** `useEffect(fn,[deps])`; `[]`=mount only; return cleanup func; `StrictMode` runs 2x in dev.

**Lists:** `{xs.map(x=><li key={x.id}>{x.n}</li>)}`.

**Stable keys, never index for dynamic.**

**Cond:** `{ok && <X/>}`, `{ok ? <A/> : <B/>}`.

**Trap:** `{count && ...}` renders 0.

**Forms (controlled):** `<input value={v} onChange={e=>setV(e.target.value)}>/`.

**Life state up:** common ancestor holds state; pass props down + callback up.

**Composition:** `children` prop; prefer composition over inheritance.

**VDOM:** diff old vs new tree → patch real DOM.

**Hook rules:** top-level only (no if/loop/func); only in components or custom hooks.

**Thinking-in-React:** 1 hierarchy 2static 3min state 4owner 5inverse flow.

**C PROGRAMMING**

**No:** classes, overloading, refs, exceptions, GC. Manual memory.

**Mem:** `malloc(n)`→ptr or NULL; `calloc`, `realloc`; `free(p)`; `p=NULL`; Failure modes: leak, use-after-free, double-free, segfault.

**Ptrs:** `int *p; &x` addr; `*p` deref; `p->f = (*p).f`. Arrays decay to pointer in args.

**Strings:** char arr ending `'\0'` (0, not `'0'`=48); `strlen/strcpy/strncpy/strncpy`. Buffer overflow if no terminator.

**const:** `const char *s` = read-only view; casting away `const` = UB.

**Structs:** `struct P{int x;};, typedef struct{...}P;` Fixed-size before variable.

**Errors:** return 0=ok, nonzero=err; output via pointer arg; `errno` global, check after libc.

**File I/O:** `FILE*f=fopen(p,"r")`; check NULL; `fread/fwrite/fgets/fprintf; fclose` on every path.

**Build:** `.C` → preprocess → compile → `.o` → link → exe. Static link = fat self-contained; dynamic = `.so/.dll` loaded at runtime.

**libc:** portability layer; same API, different impl per OS.

**Cleanup label:** single `goto cleanup`; at fn end to release resources on error paths.

**Headers:** `<stdio.h>` sys, "my.h" local; guard `#ifndef X #define X ... #endif`.

**JAVA**

**Entry:** `public static void main(String[] args)`.

**Primitive:** byte/short/int/long/float/double/char/boolean.

**Wrappers (heap):** Integer, Double, ... required for generics `List<Integer>`.

**Autobox:** int↔Integer auto. **Unbox null → NPE.** Integer cache [-128,127] makes `==` "work" then fail at 128.

**== vs equals:** `==` = identity for objects; use `.equals()`. Strings interned for literals.

**Access:** `public`, `protected` (pkg+sub), (*default*) = package-private, `private`.

**Inherit:** single class; multi-interface; `extends`+`implements` methods virtual by default; `super(...)` first in ctor.

**abstract** class = state+behavior; **interface** = behavior only (default methods OK Java 8+).

**Generics:** type erasure ⇒ `List<S>` and `List<I>` same at runtime; no `new T()`; no `instanceof List<S>`. Bounded `<T extends Comparable<T>`; wildcards `?` `extends T` (read), `? super T` (write).

**Coll:** declare to iface: `List<S> xs=new ArrayList<>(); ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap.`

**Exc:** Checked (catch or declare) vs Unchecked (Runtime/NPE/CCE). `try/catch/finally`,

**try-with-resources** `autoclose`.

**Equals contract:** override `equals` + `hashCode` together.

**GIT**

**Trees:** Working ↔ Index/Stage ↔ Repo. `HEAD` = current commit.

**Setup:** `git init`, clone URL, config `-global user.name/email: .gitignore (*.pyc, !keep.log negate)`.

**Commit:** add f, add -A, commit -m, commit -am (tracked only); `diff, diff -staged, diff HEAD`. Msg: 50/72; imperative ("Add" not "Added").

**Inspect:** `status, log, log a..b, show C, blame f, relog` (saves lost commits).

**Undo:** `restore f` (discard WT), `restore -staged f` (unstaged), `stash/stash pop, clean -fd` (untracked).

**Branch:** `branch, switch -c name, branch -d, branch -D` force.

**Merge/Rebase:** `merge B` (2-parent commit) vs `rebase B` (linear, **rewrites**); `merge -squash`. Conflicts marked `<<< HEAD ===== >>>` B; edit, add, commit.

**Remote:** `push -u` origin b, `fetch, pull, pull -rebase, remote -v`.

**Rewriting (local only):** `commit -amend; reset -soft/-mixed/-hard HEAD-1; rebase -i HEAD-3`.

**Safe undo on shared:** `revert C` (inverse commit).

**Tags:** `tag v1` (light) vs `tag -a v1 -m` (annotated); **push -tags**.

**Golden rule:** never rewrite pushed history; **-force-with-lease** safer than `-force`.

**Detached HEAD:** `switch -C new` to save commits.

**MAKE & MAKEFILES**

**Target :** prereqs then **TAB**-indented recipe (NOT spaces).

**Build:** DAG of targets; rebuild if any prereq newer than target (mtime).

**Auto vars:** `$@` target, `$<` 1st prereq, `$^` all prereqs, `$?` newer-than-target, `$(*)` stem.

**Pattern rule:** `%:o %:c`

**\$(CC) \$(CFLAGS) -c \$< -o \$@**

**Vars:** = lazy, := immediate, ?= if-unset, += append; subst `$(SRCS:.c=.o)`. Functions: `$(wildcard *.c)`, `$(patsubst %.c,%.o,$(S))`, `$(filter %.c,$(F))`, `$(foreach v,l,t),$(shell cmd)`.

**Phony:** `.PHONY: clean test all` → action, always run.

**Order-only:** `tgt: req | dir` - must exist, doesn't trigger rebuild.

**Cond:** `ifeq/ifdef/ifndef` ... `endif`.

**Flags:** `make -n` dry-run, `-j N` parallel, `-B` force, `-d` debug, `-p` print db, `VAR+=` override.

First non-special rule = default goal. Each recipe line = separate shell (chain with `;` or `&&`).

**Special:** `.DEFAULT_GOAL`, `.PRECIOUS`, `-include f` (ignore missing).

**TESTING / TDD / REFACTOR**

**Levels:** Unit (one unit, isolated) → Integration → E2E. Pyramid: many unit, few E2E.

**TDD cycle:** **Red** → **Green** → **Refactor**. Uncle Bob's 3 rules: no prod w/o failing test; just-enough test to fail; just-enough code to pass.

**V&V:** Verification = "built right?" (auto); Validation = "right thing?" (manual/customerer).

**Doubles:** Dummy (passed, unused), Stub (canned reply), Spy (record), Mock (verify expectations), Fake (light impl).

**pytest:** files `test_*.py`, funcs `test_*`, `assert x==y`; `pytest.fixture` (setup/teardown).

**Gherkin:** Given/When/Then. Playwright: `page.goto/getByRole/expect`.

**Coverage:** line, branch, path. 100% ≠ bug-free.

**Test behavior** not implementation; survives refactor.

**Refactoring:** change structure, preserve behavior. **Smells:** long fn, dup code, big class, long param list, switch-on-type, feature envy, data clumps, magic numbers, **dead code**.

**Catalog:** Extract Method/Class, Rename, Move Method, Replace Conditional w/ Polymorphism, Introduce Parameter Object.

**Failures:** Knight Capital (dead code), Ariane 5 (16-bit overflow cause), Therac-25 (race).

**Regression test:** test for the bug; keep forever.

**DEBUGGING**

**Vocab:** **Fault** = bad code; **Error** = bad runtime state; **Failure** = visible bad behavior. Flow: Fault → Error → Failure.

**4 steps:** 1 Reproduce 2 Locate fault 3 Root cause 4 Fix + verify (+ regression test).

**Methods:** Rubber duck (explain line-by-line); print/log; scientific (hypothesize, test); bisect (**git bisect**); delta debug.

**Logging:** DEBUG, INFO, WARNING, ERROR, CRITICAL. Don't log secrets.

**Assertions** document & enforce invariants near suspicious code.

**VS Code:** breakpoints (cond/log), step over/into/out, Locals/Watch/Call stack/Debug console; `launch.json` args.

**gdb:** `b file:N,r args,n/s/c,p var,bt,watch,info locals`.

**Curse of knowledge:** you read what you intended, not what's there.

**First instinct:** search the error (strip identifiers first), then ask coworker.

**Anti:** bare `except:` swallows; "works on my machine"; temp `print` committed.

SECURITY & AUTHENTICATION

**CIA triad:** Confidentiality, Integrity, Availability.  
**AuthN ≠ AuthZ:** who you are vs what you may do.  
**Principles:** Least Privilege; Zero Trust (sanitize all input); Defense in Depth; Open Design (no security by obscurity); Fail Securely; Trust Boundary.  
**Threat model:** who? what? resources? incentive? Plan = Security model + Threat model + Attack surface + Protections.  
**Common attacks:**  
· **SQLi:** `' OR "="`. **Fix:** parameterized queries (`?, @1`); never concatenate.  
· **XSS:** inject `<script>` into trusted page. **Fix:** escape output, CSP, HttpOnly cookies.  
· **CSRF:** ride logged-in session. **Fix:** CSRF tokens, SameSite cookies.  
· Path traversal `../`; command injection; RCE; SSRF; DoS.  
**Crypto:** **Sym** AES (same key); **Asym/PK** RSA/ECC (pub encrypt, priv decrypt); **Hash** SHA-256 (1-way); **Sign:** priv signs hash, pub verifies. **Salt** pw + slow hash (bcrypt/script/argon2). **HTTPS** = HTTP + TLS. TLS handshake: CA-signed cert, key exchange, symmetric session key.  
**Auth:**  
· **Session cookies:** server holds session ID; HttpOnly + Secure + SameSite.  
· **JWT:** stateless; **header.payload.signature** (base64url); store in HttpOnly cookie, **not localStorage**.  
· **OAuth2:** delegated auth; **OIDC** = OAuth + identity layer.  
**Passwords:** never plain or unsalted hash; bcrypt/argon2.  
**OWASP Top:** broken access ctrl, crypto failures, injection, insecure design, misconfig, vulnerable deps, auth fail, SSRF.

NETWORKING / CLIENT-SERVER

**Architecture:** Client-Server (centralized) vs P2P (decentralized; producer+consumer). Hybrid e.g. Zoom.  
**Throughput** = vol/time; **Latency** = time/req. Scale: vertical vs horizontal.  
**TCP/IP stack (top→bot):** 1 Application (HTTP/HTTPS/SSH/DNS/FTP/SMTP) 2 Transport (TCP/UDP) 3 Internet (IPv4/v6) 4 Link (Ethernet/WiFi/MAC).  
**TCP** reliable, ordered; 3-way handshake SYN → SYN-ACK → ACK; teardown FIN. **UDP** fire-and-forget; streaming/games/DNS.  
**IP:** v4 32-bit (4 octets), v6 128-bit; **127.0.0.1** = localhost. Ports 0–65535; well-known <1024 (80, 443, 22, 53).  
**DNS** maps name → IP. **HTTPS**=443, **HTTP**=80.  
**HTTP:** stateless req/resp. Verbs:  

| Verb                 | Safe | Idemp. |
|----------------------|------|--------|
| GET / HEAD / OPTIONS | yes  | yes    |
| PUT / DELETE         | no   | yes    |
| POST / PATCH         | no   | no     |

  
**Status:** 1xx info; **2xx** 200/201/204; **3xx** 301/302/304; **4xx** 400/401/403/404/409/429; **5xx** 500/502/503/504.  
**Headers:** Content-Type (application/json), Authorization, Cookie, CORS (Access-Control-Allow-Origin), Cache-Control.  
**URL:** `scheme://host:port/path?query#frag`.  
**WebSocket:** full-duplex over TCP after HTTP upgrade.  
**CORS:** browser same-origin policy; server allows cross-origin via headers; preflight OPTIONS for non-simple requests.  
**Inventors:** Kahn & Cerf (Cerf @ UCLA).

CLEAN CODE

**4 Pillars:** Meaningful Naming, Simplified Structure, Purposeful Docs, Design by Contract.  
**Naming:** wrong name > meaningless > vague > precise short. Classes = nouns; methods = verbs. `snake_case` Python, `camel-case` JS/Java/C++.  
**Structure:** guard clauses (early exit); avoid deep nesting; short funcs (1 screen); 5–9 chunks (Miller).  
**Comments:** **Why**, not **what**. Misleading > outdated > absent. Self-documenting names beat comments. Refactor before commenting.  
**DbC:** pre-cond, post-cond, invariant. **assert** enforces; **-DDEBUG** strips.  
**Principles:** DRY, KISS, YAGNI, SoC, POLA (least astonishment), Law of Demeter (don't talk to strangers; **a.b.c.d** smell), Boy Scout/Scout Rule (leave cleaner).  
**SOLID:**  
· SRP: one reason to change.  
· OCP: open for extension, closed for modification.  
· LSP: subtypes substitutable.  
· ISP: small client-specific interfaces.  
· DIP: depend on abstractions.  
**Stats:** Read:Write = 10:1 to 14:1; 58% dev time = understanding.

MANAGING COMPLEXITY

**Essential vs Accidental** complexity (Brooks, “No Silver Bullet”).  
**Big Ball of Mud:** low modifiability, low understandability, high fragility.  
**Information Hiding (Parnas):** list likely-to-change decisions; each module hides one. Interface = stable WHAT; impl = changeable HOW.  
**Deep vs Shallow modules (Ousterhout):** small iface hiding large functionality = GOOD; wide iface, thin impl = BAD.  
**Coupling LOW · Cohesion HIGH.** Coupling: data < stamp < control < common < content. Cohesion: functional > sequential > communicational > procedural > temporal > logical > coincidental.  
**Syntactic** dep (compiler catches) vs **Semantic** (sneaky; same design decision).  
**Single Choice / DRY:** only one module knows the full alternatives list.  
**Change Impact:** if likely + costly → redesign.  
**Design process:** identify reqs → generate many alternatives → evaluate → trade off → decide. Broaden in generate, narrow in evaluate.  
**Delay decisions** until pressure forces them; solve simpler problem first.  
**Design doc:** Context & Scope · Goals/Non-Goals · Design · Alternatives.

CODE COMPREHENSION

**Bottom-up** (novice): line-by-line chunking. **Top-down** (expert): hypothesis-driven, scan for **beacons**.

**Cycle:** hypothesize → hunt beacons → verify/reject → revise; toggle modes opportunistically.  
**5 Beacons:** identifier names, code structures (loop/recursion shape), tests, assertions, diagrams.  
**Build domain knowledge** first (reqs/architecture).  
58% of dev time = understanding code. Design modules usable without reading internals.

CODE REVIEW

**Expectation:** find bugs. **Reality:** knowledge transfer, maintainability – only ~15% defect comments (Bacchelli & Bird).  
**Top categories:** naming 22%, organization 16%, alt solutions 9% → 50% maintainability.  
**Effective PRs:** 1–2 reviewers (more = bystander); 200–400 LOC; quality drops after 1 hr; stack small PRs vs bundle.  
**Purposes:** readability, teach juniors, norms, history, gatekeeping, accident prevention.  
**Checklist:** 4 Pillars + correctness + tests + security + performance.  
**Scout Rule:** leave cleaner; verify nothing hidden.  
New reviewers need 6–12 months to be productive.

DESIGN & MODELING (UML)

**4 views:** Module (Class diag) · Data (ERD) · Component-Connector (runtime) · Behavioral (Sequence/State).  
**Class:** + pub, – priv, # prot. **Inheritance** = open white Δ. **Assoc** = line (nav arrow). **Aggregation** = open o (has-a). **Composition** = filled ♦ (lifetime). **Multiplicity** 1, \*, 0..1, 1..\*.  
**ERD:** rectangle = entity, o = relation, oval = attribute. Cardinality 1:1, 1:N, N:M.  
**Sequence:** solid ► sync call; dashed ▷ return; open ► async. Activation bar = executing. Frames: **alt** (if/else), **opt**, **loop**, **par**.  
**State:** rounded states + arrows **[guard]** event / action; initial = ●.  
**Has-a vs Is-a:** prefer composition over inheritance.

DESIGN PATTERNS (GOF)

**Definition:** Context · Problem · Solution (roles) · Consequences. Pattern = template, not code.  
**Creational:**  
· **Singleton** – 1 instance globally accessible.  
· **Factory Method** – subclass picks class to instantiate.  
· **Abstract Factory** – families of related objects.  
· **Builder** – step-by-step construct complex object.  
· **Prototype** – clone existing object.  
**Structural:**  
· **Adapter** – translate one iface to another (XML↔JSON).  
· **Decorator** – wrap to add behavior dynamically.  
· **Facade** – simple iface over subsystem.  
· **Proxy** – stand-in (lazy/access/remote).  
· **Composite** – tree of part/whole uniformly.  
· **Bridge** – decouple abstraction from impl.  
· **Flyweight** – share many small objects.  
**Behavioral:**  
· **Observer** – 1 Subject:N Observers; auto **update()** (GUI, signals).  
· **Publish-Subscribe** – N:M via named topic + broker (async; generalizes Observer).  
· **Strategy** – swap algo via composition; supports OCP.  
· **Command** – action as object (undo, queue).  
· **Iterator** – sequential access w/o exposing internals.  
· **State** – change behavior when internal state changes.  
· **Template Method** – skeleton in base, hooks in sub.  
· **Chain of Responsibility** – pass req along handlers.  
· **Mediator** – central coordinator (cut  $N^2$  coupling).  
· **Visitor** – operation across object structure.  
· **Memento** – capture/restore state.

DESIGN WITH REUSE

**Why:** productivity, faster TTM, higher quality, fewer defects.  
**Reality:** partial fit; glue code needed.  
**Internal** (same org/product line) vs **External** (COTS/OSS/framework).  
**Library** = you call it. **Framework** = it calls you (Inversion of Control: “don’t call us, we’ll call you”). Framework = less flex, less code.  
**White-box** reuse (inherit) vs **Black-box** (compose).  
**Dependency Injection:** pass collaborator in, don’t **new**; supports DIP + tests.  
**External rules:** pin versions (Pipenv/lockfiles); update regularly for CVEs; minimize deps; fit-to-context > popularity.  
**Identify violated assumptions:** **Ariane 5** reused 16-bit horizontal-velocity int → overflow → \$370M lost in 37s.  
**Cost/benefit:** save impl + test vs integration + update + limited changeability.  
**Generic-Specific:** parameterize, hooks, plugins.

API DESIGN & INTEROPERABILITY

**Interop** = exchange + use meaningfully. **Syntactic** (format/types) vs **Semantic** (units/vocab/side-effects).  
**Mars Climate Orbiter** (\$193M): lbf vs N – same syntax, different semantics.  
**REST:** stateless; HTTP verbs as CRUD (POST/GET/PUT/DELETE); resource URLs `/users/123`; JSON/XML.  
**vs RPC** (stateful, action) · **SOAP** (XML, complex, enterprise) · **GraphQL** (schema, client picks fields).  
**Idempotency:** same effect repeated. **Safety:** no side effects.  
**Versioning:** `/v1/` in URL or header. Backward-compat adds OK; removals = major bump.  
**Schema** (JSON/XML/YAML): types · values (min/max/enum) · structure (required). Enables auto validation.  
**Serialization:** JSON (web), XML (enterprise/SOAP), YAML (config), Protobuf (binary, gRPC), CSV.  
**Doc both views:** syntactic (params, fmt) + semantic (units, side-effects, errors, examples).  
**Extensible iface:** add fields without breaking clients (ignore unknown).  
**Adapter pattern** localizes translation.  
**Trade-off:** richer iface = more flex but harder adoption.

DATA MANAGEMENT

**DBMS** handles concurrency, recovery, indexing, optimization.

**SQL** = declarative (WHAT not HOW).  
**ERD → Relations:** entity → table, attr → column, relation → FK or junction table (M:N).  
**Keys:** Primary (unique, NOT NULL); Foreign (refs PK, integrity); Composite; Surrogate.  
**SQL skeleton:**  
`SELECT c, COUNT(*) FROM t WHERE x>0 GROUP BY c HAVING COUNT(*)>1 ORDER BY c DESC LIMIT 10;`  
**Joins:** INNER (matching), LEFT/RIGHT (keep one, null other), FULL OUTER, CROSS (cartesian).  
**Normalization:** 1NF (atomic), 2NF (no partial dep), 3NF (no transitive dep), BCNF.  
**ACID** (RDBMS): Atomicity, Consistency, Isolation, Durability.  
**BEGIN/COMMIT/ROLLBACK.**  
**CAP (pick 2):** Consistency, Availability, Partition tolerance. RDBMS = CA; Cassandra = AP; MongoDB = CP.  
**NoSQL:** Document (Mongo), Key-Value (Redis), Wide-Column (Cassandra), Graph (Neo4j). Eventual consistency; horizontal scale; flexible schema.  
**Index** speeds reads, slows writes; B-tree default.  
**SQLi defense:** parameterized queries always.

CODING WITH GENAI

**LLMs:** pre-train (corpus) → post-train (RLHF) → inference (non-deterministic). “Statistical parrots” – hallucinate APIs, repeat outdated/insecure code.  
**Risks:** 40% of Copilot suggestions have vulns; Cursor study: +30% warnings, +42% complexity; “skill atrophy” via cognitive offloading.  
**Amplifier (DORA):** AI boosts high-perf teams (CI/tests/clean arch); magnifies dysfunction in struggling ones.  
**Best practices:**  
· Mandatory review; treat AI as unreliable teammate.  
· Never commit code you can’t explain.  
· AI = knowledgeable intern; YOU architect.  
· Pair: AI as driver/navigator.  
· TDD pairs well with AI (test = verification).  
**Good use:** scaffolding, refactor, docs, simple debug, prototyping.  
**Bad use:** business logic, security-critical, performance-critical, stateful complex flows. “Vibe coding” = hobby only.  
**Prompt structure:** **Role** + **Task** + **Context** + **Instructions** + acceptance criteria. Split big tasks; replace-TODO pattern.  
**Don’t ask** “should I use State pattern?” → ask “what are its trade-offs?”

PEOPLE & PROCESSES

**Paradigms:**  
· **Waterfall** (plan-driven): sequential phases; stable reqs, high cost of change.  
· **Agile/Scrum:** iterative sprints; embrace change.  
· **Risk-driven (Fairbanks):** upfront design ∝ risk.  
**Agile Manifesto:** individuals, working sw, collab, response to change > processes, docs, contracts, plan.  
**Scrum:** Product Backlog → Sprint Planning → Sprint (2–3wk) → Daily Standup → Sprint Review → Retro. Roles: PO, Scrum Master, Dev Team. Planning poker = effort, not time.  
**INVEST** stories: Independent, Negotiable, Valuable, Estimable, Small, Testable.  
**Risk-driven:** identify hard-to-change decisions; risk storming; high cost change → more design; preserve changeability via Info Hiding/SOLID.  
**Tech debt backlog:** dedicate sprint capacity; refactor + docs + perf.  
**Decisions:** Rational (explicit, justifiable, slow) vs Intuitive (fast, expert, hard to justify). Biases: anchoring, confirmation, bounded rationality.  
**Anti-patterns:** Ivory Tower architect; one-size-fits-all process.  
**Scale:** Doghouse vs Skyscraper; right-size the process.  
**Adversarial thinking:** how could this be abused?  
**Archetypes:** Meta = fast/break; Google = design docs; NASA = plan-driven V&V; Startup = lean MVP.

QUICK CROSS-REFERENCES

**Idempotent verbs:** GET, PUT, DELETE, HEAD, OPTIONS. *Not:* POST, PATCH.  
**Safe verbs:** GET, HEAD, OPTIONS.  
**Symbols overload:** **\$@** = target (make) vs all args (bash); **?\$** = newer-than-target (make) vs exit status (bash).  
**Equality:** `Py is None`, `JS ==`, `Java .equals()`, `C strcmp`, `Bash = (str) / -eq (num)`.  
**Acronym order:** Pillars = Naming-Structure-Docs-Contracts; CIA = Confid-Integ-Avail; CAP = Consist-Avail-Partition; ACID = A-C-I-D; SOLID = S-O-L-I-D; INVEST = I-N-V-E-S-T.  
**Famous fails:** Mars Orbiter (units), Ariane 5 (int overflow reuse), Knight Capital (dead code), Therac-25 (race), Twitter onmouseover (XSS), BA 2018 (XSS, 250K cards).